Chapter 7
Hypotheses Engineering in Software Startups: From Business to Architecture



Eduardo Guerra, Jorge Melegati, and Davide Montesin

Abstract Software startups operate in environments marked by high uncertainty and rapid innovation, yet many still prioritize technical excellence over validated customer needs. This chapter advocates for a shift toward hypotheses engineering: a systematic approach to uncover, test, and manage assumptions in startup development. We present an integrated application of two complementary techniques: HyMap, which supports early-stage identification and mapping of business hypotheses, and ArchHypo, which manages architectural uncertainties by treating them as hypotheses. Through a case study of a real-world startup integrating Generative AI into their product, we demonstrate how connecting business and architectural hypotheses enable traceable, strategic decision-making. The results show that combining HyMap and ArchHypo provides a more holistic, hypotheses-driven development process, improving a startup's ability to adapt, experiment, and align technical efforts with business goals.

Keywords Hypotheses engineering · Software architecture · Business assumptions · Software startups

7.1 Introduction

Software-enabled innovations that changed human life are ubiquitous in contemporary society. Web search engines, e-commerce Web sites, and social networks are just some examples. Many of these solutions were developed by emerging companies, the so-called startups. When the product or service is software intensive, these companies are called software startups [36]. Even though there are household

E. Guerra · J. Melegati (✉)
Faculty of Engineering, Free University of Bozen-Bolzano, Bolzano, Italy
e-mail: eduardo.guerra@unibz.it; jorge.melegati@unibz.it

D. Montesin
Catch Solve, Bolzano, Italy
e-mail: d@vide.bz

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

N. Tripathi et al. (eds.), *Advances in Software Startups*,
https://doi.org/10.1007/978-3-032-04294-1_7

names, e.g., Slack, Airbnb, and Instagram, representing successful software startups, more than 90% of startups fail [18]. There are various reasons for failure [36], but the lack of a structured business development process is a key determinant [6]. A key practice that could support these companies, by reducing the uncertainty involved in the process, is experimentation [1, 11]. This approach is characterized by continuously identifying critical product assumptions, transforming them into hypotheses, prioritizing, and testing them with experiments following the scientific method in order to support or refute them [20]. However, research has shown that the adoption of experimentation by software startups is still limited [17] because of an overfocus on technical aspects without considering the real needs of customers [23]. To tackle this issue, in previous work [22], we proposed the term "Hypotheses Engineering" to describe techniques to handle hypotheses in experiment-driven contexts, moving the focus from requirements.

Since then, several techniques have been proposed in the context of hypotheses engineering, not only for startups but also for consolidated companies. However, these techniques have been proposed for specific scenarios and steps of the development, still lacking a more comprehensive approach. Besides that, it has been observed that experimentation in software development has focused on what to develop, being closely linked to business development and customer development [20]. In this chapter, we focus particularly on HyMap, a hypotheses elicitation targeted to startups, and ArchHypo, a technique to handle uncertainties in software architecture by converting them into hypotheses and managing them throughout the development process. By combining these techniques, we aim to propose a full-fledged technique to support software startups with both business and architectural aspects. To evaluate our approach, we present the application of the techniques in a real software startup scenario.

The contribution of this work is the connection between HyMap and ArchHypo, proposed in Sect. 7.6, by extracting technical hypotheses related to the software architecture from the business hypotheses of the startup. Additionally, the proposed approach also suggests how to keep the related hypotheses connected, keeping the traceability among them and enabling the collaboration between the business and technical teams to handle the uncertainties. Another contribution is a case study, presented in Sect. 7.7, demonstrating the combined use of both techniques in a real scenario. It focuses on a startup that envisions the integration of Generative AI in its product, which is a scenario that captures what is happening in several other companies. In this sense, the way that the uncertainties were identified and handled can also be used as a reference in other cases. The originality of the work presented in this chapter lies in the consolidation of an approach that connects the uncertainty management, from the business to the technical hypotheses, which is not present in any technique proposed in the existing literature.

7.2 Experimentation in Software Startups

Despite being often used in research and practice, the term *software startup* does not have a clear definition [4, 29]. Based on a systematic mapping study (SMS) on the topic, Berg et al. [4] concluded that the most used terms in research to describe these organizations are innovation and uncertainty. Therefore, we will use the following definition throughout this chapter: software startups are companies that develop innovative, software-intensive products or services searching for a sustainable and scalable business model [36].

A natural objective for a startup endeavor is to be successful. In this context, a logical approach would be to distinguish the "technical" success of the software project, as a necessary condition, from the business success [21]. However, even the success or failure of a software project is a complex argument, as the literature shows. Ralph and Kelly [30] investigated the success concept in the software engineering context. First, they observed how the understanding of this concept evolved in project management literature. For a long time, a model used to describe such a phenomenon was the Iron, or Project, Triangle [30]. This metaphor postulates that the quality of a product (inner of the triangle) is constrained by three vertices: scope, budget, and schedule [2]. Then, the authors described a more comprehensive taxonomy proposed by Shenhar et al. [32], which describes success through four dimensions: project efficiency, impact on the customer, business success, and preparing for the future. Finally, the authors performed an empirical study with 191 design professionals, of whom 68 were in the software industry. The results indicated that practitioners understand software engineering success "as a multidimensional variable comprising project efficiency, artifact quality, market performance, and stakeholder impacts over time." This result is in line with findings in the information systems literature (e.g., [10]).

Therefore, software project success should not only concern the quality, scope, and schedule of the software system delivered. Following Shenhar et al.'s taxonomy, other measures that seem especially important for software startups are, for instance, fulfilling customer needs, commercial success, and creating a large market share. As Ralph and Kelly [30] observed in the discussions on software project development success, practitioners perceive the existence of a client who has problems to solve or needs to be fulfilled. Differently, software startups have to find these problems and even customers. Some types of pivots observed in software startups support such an argument. For instance, a customer segment pivot is a shift from one customer segment to another [3]. Zoom-ins are pivots where a single feature becomes the whole product, and as the opposite, zoom-out is when a product becomes only a feature of a bigger product. Consequently, software engineering in software startups, especially in the early stage, should focus on the overall goal of understanding how to apply its proposed innovation into a sustainable business model. Given the importance of software to the business model of a software startup, it is hard to separate software development from business in this context. Software development will inform how the company can create an innovative business model for a chosen

customer or market segment. Meanwhile, the business model is constrained to what a software system can perform. In summary, especially in the context of software startups, business and technical successes are mingled together. The success of the project does not only regard technical aspects but also business development, which is associated with several uncertainties, such as whether they are able to fulfill real needs from customers. Therefore, to increase their chance of success, software startups need to handle and minimize uncertainties.

A common way to deal with these uncertainties has been experimentation, which is, for example, a key aspect of the Lean Startup, the most well-known startup development methodology [5, 14]. In the context of software engineering, experimentation is defined as a process of continuously validating product assumptions, transforming them as hypotheses, prioritizing, and testing them following the scientific method in order to support or refute them [20]. This notion comprises several techniques like iterations with prototypes, gradual rollouts, and controlled experiments [12] or even problem and solution interviews [20]. Therefore, in this context, the word "experiment" is not bound to the scientific meaning but points to a broader concept of data-driven rather than opinion-based decision-making [20].

Despite the scientific results pointing to the advantages of using experimentation and industry practices, such as the Lean Startup, research has shown that its adoption in these companies is limited [17, 23]. Instead, founders and other team members consider their ideas validated and focus on implementing them [17, 23]. It has also been observed that technical aspects were mostly overcome, and the major inhibitors for the adoption experimentation are at the individual level [20, 23]. To tackle this issue, Melegati et al. [22] proposed the term "hypotheses engineering," which will be described in detail in the next section.

7.3 Hypotheses Engineering

In the literature, there are models to describe how teams execute experimentation (e.g., HYPEX [27] and RIGHT [13]). Both models emerged from multiple case studies in software development companies applying experimentation. The models present a similar sequence of steps summarized as [22]:

1. Based on the current understanding of the customer, user, and market, identify, specify, and prioritize hypotheses.
2. Design an experiment to evaluate the hypothesis.
3. Execute the experiment and collect data.
4. Analyze the results and, based on the acquired learning, update the hypotheses and return to the first step.

Based on a parallel to requirements engineering (RE) in conventional software development, Melegati et al. [22] called this set of activities hypotheses engineering (HE), to stress the importance of properly handling hypotheses in experiment-driven software development.

Experimentation is strongly related to software development, and many experiments are pieces of software themselves. In other words, experimentation is a type of software development where the software produced is intended to evaluate previously defined hypotheses. Therefore, similar activities to those in conventional RE could be useful to handle hypotheses. In our view though, HE is not a substitute for RE but, instead, one activity, part of the experimentation process, to reach the final goal of defining software requirements. A hypothesis can be considered a type of higher-level requirement to guide experiments to understand the customer and market and, finally, reach requirements for the final software system. In other words, hypotheses are requirements for experiments, and since many of these are pieces of software themselves, using similar activities for hypotheses is reasonable. Nevertheless, since there is a shift from eliciting the customer needs in conventional RE to finding which product assumptions should be evaluated, RE's current practices and techniques may not be useful.

7.4 HyMap

In the context of RE, elicitation's primary goal is to define the requirements to reach the purposes for which a software system is intended [7]. For HE, a similar activity would identify on which assumptions a product is being built and transform them into hypotheses. These hypotheses can then be evaluated, being supported or refuted by an experiment, increasing the knowledge to support the decision-making process regarding the product and its features. HyMap [24] is a hypotheses elicitation technique developed for early-stage software startups. It is based on the Personal Constructs Theory by Kelly [19], which states that a person makes sense of the world through patterns or templates, the so-called constructs, created by the person. These constructs are also used to predict and control the course of events. HyMap's goal is to make these constructs evident and use them to identify the hypotheses to be tested. To this aim, HyMap defines a visual language to depict a map representing the constructs and their relationships and a process, based on an interview with founders or other decision-makers in a software startup, to build this map and derive hypotheses based on it. Figure 7.1 describes HyMap's visual language. The advantage of HyMap to alternatives, such as Assumption Mapping, is its scientific basis, i.e., Kelly's Personal Constructs Theory, and systematic development and evolution [24].

The creation of a HyMap starts by adding elements representing the product (on the bottom of the map in Fig. 7.1) and the foreseen customers (on the top of the map in Fig. 7.1). Right below the customers, boxes representing the customers' problems to be solved or mitigated by the proposed solution are added and linked with arrows to the respective customers. Similarly, above the representation of the product, boxes are added, representing the features that are expected to contribute to the solution of the customers' problems. These features are linked not only to the product but also to the respective problems. Then, in an iterative process, the interviewee, with the

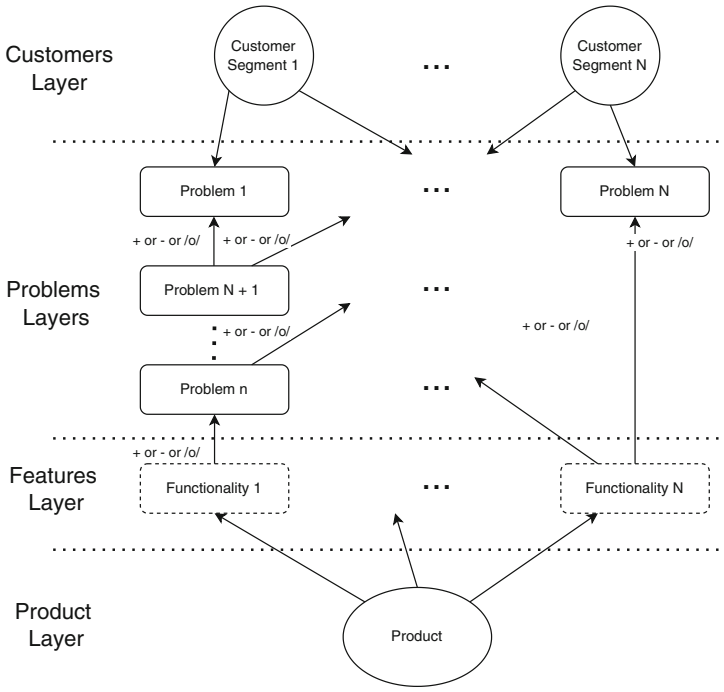


Fig. 7.1 HyMap's visual language

support of the facilitator, should evaluate each arrow to analyze if implicit concepts are used to explain that relationship. In this case, a new box should be included, which should be connected with arrows to the original boxes. The process should also be conducted on the arrows included in this step. The process ends when no new arrows are needed.

Once the map is complete, it is possible to create a hypothesis for each arrow. The elements being connected by the arrow will then define the type of the hypotheses. Arrows connecting the product to its features lead to "product hypotheses," which regard the team's capability of implementing that feature from a technical perspective. These hypotheses can be formulated using the following template: "the team developing <the product> is capable of implementing <the functionality>." Arrows connecting the features to problems or problems among themselves lead to "value hypotheses," which regard the relationship among different problems and problems with the features. These hypotheses can be formulated using the template: "<the functionality or the problem> <increases/decreases/ or has no effect> on <the problem>." Finally, arrows connecting problems to the customers lead to "problem hypotheses," which regard whether those are relevant problems for the customers. These hypotheses can be formulated using the template: "<The customers or a segment of them> <has/would like to solve> <the problem>."

7.5 ArchHypo

ArchHypo [34, 35] is a technique based on hypotheses engineering for managing uncertainties in the software architecture. It is based on a set of patterns for agile architecture [15, 37], in which the hypotheses are used as an implementation strategy for the pattern FIND WHERE IT HURTS,[1] based on the premise that architecture design should focus on points with higher uncertainty. The hypotheses can be stated as clear, straightforward assertions that reflect the development team's current assumptions.

After identifying the architectural hypotheses, each one should be assessed by the development team in terms of its level of uncertainty and its impact [33]. The level of uncertainty indicates how close the team is to either validating or disproving the hypothesis, while the impact reflects the potential consequences and costs of switching to an alternative solution. These two assessments are independent and are intended to encourage reflection on the hypothesis. They support a qualitative comparison with other hypotheses and assist in selecting appropriate techniques for addressing it. Since the goal is not to measure these characteristics with precision, a simple rating scale, such as five levels (Very Low to Very High), can be used. After this initial classification, the hypotheses are reviewed in each iteration, evaluating if they were accepted or rejected and, in case not, readjusting the uncertainty and impact assessment.

To handle the hypotheses, the team should elaborate a technical plan with actions to reduce uncertainty or impact. On the one hand, using techniques to obtain more knowledge about the hypothesis would reduce the uncertainty; on the other hand, introducing flexibility or modularization can reduce the impact. The technical plan is an implementation strategy for the pattern PLAN FOR RESPONSIBLE MOMENTS [15], and its action should be added to the project backlog to be prioritized and executed. The connection with the hypothesis should be considered when its prioritization is being discussed. The plan may consist of one or more actions intended to (a) conclusively confirm or reject the hypothesis; (b) decrease the level of uncertainty; (c) lessen the potential impact; or (d) establish criteria for deferring its resolution.

As an example, patterns related to software analytics [9] can be used to reduce uncertainty, and a PROTECTIVE GUIDELINE [28] can introduce coding practices and constraints to reduce the impact. It is important to highlight that the goal is not to solve all the hypotheses as soon as possible, since uncertainty is inherent to software projects, but to manage them to evaluate the right moment to make architectural decisions. In this sense, the technique provides a more systematic way to evaluate decisions that can be postponed, which is an important practice associated with continuous architecture [38].

[1] Pattern names are highlighted in small caps font, with capital letters.

ArchHypo is aligned with the idea that good solutions come with combining technical expertise with creativity [16, 25]. So, while these patterns present a set of practices for handling architectural hypotheses, the teams should not be limited to them and should also search for creative alternatives.

7.6 Building a Bridge Between the Techniques

On the one hand, HyMap produces a set of hypotheses related to the targeted customers, their problems, and if the product solves or mitigates the problems. On the other hand, ArchHypo's starting point is a set of hypotheses related to the software architecture. While the connection between the techniques might seem that it is just using the hypothesis identified by one technique and managing in the other, not all hypotheses found in HyMap might affect the architecture and be suitable to be handled in ArchHypo. In some cases, the connection or implications of the hypothesis might not be explicit. This connection must be considered in two different moments: (1) when the hypotheses are identified and (2) when creating a strategy for handling and managing the hypotheses. The next subsections go deep into each of these aspects.

7.6.1 Hypotheses Elicitation

When using HyMap, different types of hypotheses are generated after creating the map, considering the arrows from different layers in the model. Different implications for the software architecture might arise for each kind of hypothesis. Figure 7.2 presents the type of architectural hypotheses that would be generated, and the following explains the rationale of each one:

- **Product hypotheses** (*based on arrows from the product to the features layer*): This type of hypothesis can generate architectural hypotheses related to feasibility and team knowledge. In the first case, it should represent the uncertainty regarding the technical viability of implementing that functionality. In the second case, the hypotheses would be related to the team's lack of knowledge of the technology that would enable that functionality. In other words, by other means, it is possible to know the feasibility, but with some unknown consequences in other quality attributes. *Example: Consider the hypothesis "the team developing <product> is capable of implementing a 3D preview based on customer description." A feasibility hypothesis would represent the uncertainty related to the viability of implementing the feature. If the team does not have experience with the related technologies, other hypotheses can represent the uncertainties related to that, such as the impact on performance and the integration in a Web environment.*

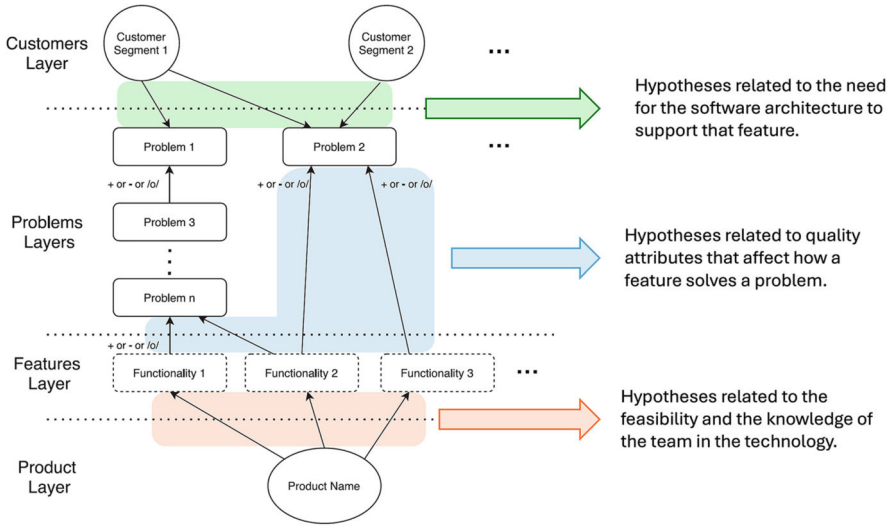


Fig. 7.2 Extracting the hypothesis related to the software architecture from the HyMap conceptual map

- **Value hypotheses** (*based on arrows from the features layer to the problem layers*): These hypotheses are related to the influence that the proposed features have on the problems. In this case, the hypotheses related to the software architecture are not direct but related to important quality attributes that might influence this causal relationship. In this case, the architectural hypothesis will be related to the influence of this requirement. *Example: Consider the hypothesis "importing the user's medical profile decreases the problem of the lack of information to make decisions." Given existing privacy regulations, security is an important quality attribute that would enable this causal relationship to be possible. Since the team has no knowledge about the security protocols that need to be implemented, an architectural hypothesis would represent this uncertainty.*
- **Problem hypotheses** (*based on arrows from the customers' layer to the problem layers*): These hypotheses are related to the necessity and importance of a given problem for the customers. The impact on the architecture happens if this feature has some influence on an important quality attribute, since the decision to implement it would depend on the hypothesis evaluation. *Example: Consider the hypothesis "<customer> has to follow the current state of the sensors managed by the system." This hypothesis is important to determine quality attribute requirements. Since accessing the current state of the system sensors would require receiving real-time information, performance, and availability requirements would be influenced by this hypothesis. In this case, the architectural solutions would be very different based on the hypothesis acceptance or rejection.*

It is important to highlight that not all business hypotheses have a respective architectural hypothesis. For instance, the addition or absence of functionality that can be strategic from the business side might not impact the software architecture. Similarly, there might also be some uncertainties in the software architecture that are directly related to the technologies used and are not directly connected to the business side. An example would be related to comparing design solutions to optimize a given quality attribute that is not related to a business uncertainty.

After the identification of the architectural hypotheses, ArchHypo should proceed with the elaboration of the technical plan. Some hypotheses related to the customer needs cannot have their uncertainty level reduced from a technical point of view. However, some actions can be done to reduce the impact. In this case, the usage of techniques that can add more flexibility in the architecture, such as PROTECTIVE GUIDELINE [28], can also provide more flexibility in the startup business, allowing decisions to be postponed without serious consequences to the software architecture.

7.6.2 Hypotheses Management

Even if the architectural hypotheses are more specific and related to technical aspects, keeping traceability between them and the business hypothesis is important. In this case, they can be different, but the source of uncertainty is the same. The strategy built to handle the uncertainty on the business and architectural sides should consider each other since one can highly influence the state of the other.

For instance, when the uncertainty on the business side is reduced, this fact should trigger a reduction in the uncertainty level of the respective hypothesis. On the other side, if an action reduces the impact related to a hypothesis in the software, it also reduces it on the respective business hypothesis. This connection can be used strategically by the startup to use actions on one side to support the other, as presented in Fig. 7.3.

As one example, when an important business hypothesis is difficult to evaluate in the short term, the technical plan to handle the respective architectural hypothesis might seek to reduce its impact. That will provide more flexibility on the business side, giving more time to explore the possibilities and make the decision. Accordingly, with a reduced impact, the risk of experimenting with one approach is reduced since it would be easier to change to the other. In this sense, an architectural solution can support the business by giving it more freedom to explore.

For an architectural hypothesis with a high impact, experimentation aiming at the reduction of the uncertainty from its business part can help to support technical decisions. By understanding where it is easier to have flexibility in the software architecture, the startup can prioritize actions to reduce the business uncertainties where it would have a larger impact.

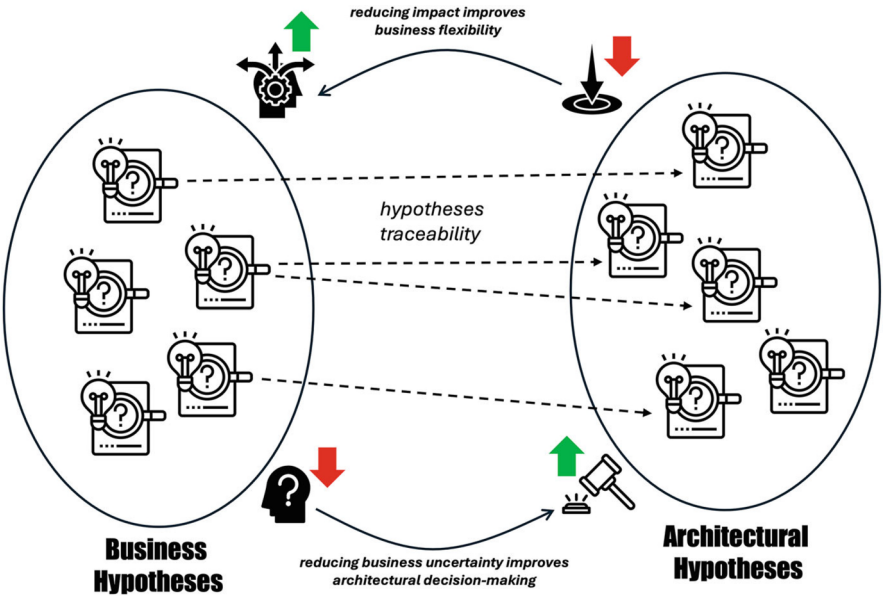


Fig. 7.3 Strategic actions in business and architectural hypothesis management

7.7 Case Study

To demonstrate the proposed technique, we verified how the combination of HyMap and ArchHypo would perform in the real context of a software startup called Catch Solve. This case was chosen because the startup owner is one of the authors of this chapter. Their product focuses on monitoring and testing Web applications, providing features to periodically run automated tests simulating real users to verify the functionality and availability of a Web application. One of the highlights of their platform is the functionality of providing reports with a screen recording of any error found. This startup already has previous experience in using hypotheses for architectural uncertainties [35].

7.7.1 Context and HyMap Application

The context considered is the possibility of incorporating artificial intelligence into the startup product. Given the rise of Generative AI [8, 31] and its potential in applications for software engineering [26], the startup owner considered that it would be strategic to integrate it into their platform. The integration would also open possibilities for participation in calls for funding related to the application of artificial intelligence. His idea was to have a conversational interface in which the

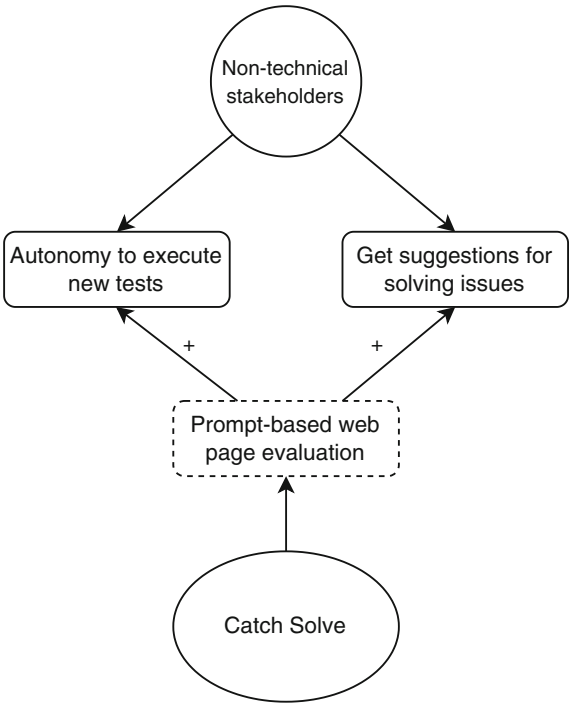


Fig. 7.4 HyMap for the use of Generative AI in Catch Solve

users could interact with a model in a way that it could invoke Catch Solve when appropriate. However, there was a lot of uncertainty related to the feasibility of that idea, how it could be implemented, and the consequences to the current software architecture.

To identify the hypotheses related to this change in the product, we developed a HyMap focused on this aspect, i.e., not representing the whole product but specifically the addition of AI to the product. The result of this exercise is presented in Fig. 7.4. The customer's need of *"autonomy to execute new tests"* comes from the fact that, in the current version of the product, all the tests need to be added by the startup, and allowing clients to create new tests would give more freedom to them and contribute to the business scalability. The other need represented in the figure, i.e., *get suggestions to solve issues*, depicts a functionality envisioned from the beginning of the product that had not been implemented yet.

Based on the generated HyMap (Fig. 7.4), we developed the following business hypotheses:

- BH1. Non-technical stakeholders want autonomy to execute new tests.
- BH2. Non-technical stakeholders want to get suggestions for solving issues.
- BH3. A prompt-based Web page evaluation would increase the autonomy to execute new tests.

– BH4. A prompt-based Web page evaluation would facilitate getting suggestions for solving issues.
– BH5. The Catch Solve development team is capable of implementing prompt-based Web page evaluation.

Since the product has been developed for some period of time, the team considered that it had enough data collected through interactions with customers to support the two problem hypotheses, namely, BH1 and BH2. Since the tests that monitor the Web application need to be developed by the startup team, BH1 would be important for product scalability. As stated in the startup name, Catch Solve, one of the initial goals was to already provide solutions to the problems detected. In this direction, even if currently the product only has the functionality of finding errors, BH2 is aligned with the initial startup vision. So, no architectural hypotheses related to these two hypotheses were added.

7.7.2 Translation into Architectural Hypotheses

Considering the value hypotheses, BH3 and BH4, it was evaluated if there is a quality attribute that could influence this causal relationship. In this case, reliability was pointed out as an important attribute since the product's APIs should be invoked appropriately, and the suggestions should make sense. The product hypothesis, BH5, was also considered in terms of the feature feasibility. Considering the intent to evaluate the usage of Generative AI in this context, the architectural hypotheses were also elaborated to express the assumption of the suitability of this technology. Based on this rational, the following architectural hypotheses were elaborated:

– AH1. Using Generative AI, the product APIs can be reliably invoked based on user prompts. (*mapped to BH3*)
– AH2. Using Generative AI, reliable suggestions can be given based on the problems found. (*mapped to BH4*)
– AH3. A prompt-based Web page evaluation can be implemented using Generative AI. (*mapped to BH5*)

Following ArchHypo, the next step was elaborating a technical plan to deal with the hypotheses. Since the startup did not have previous experience with Generative AI, they decided to start with an ARCHITECTURAL SPIKE [37] to handle AH3. The goal of it was to evaluate the ways in which the application could interact with the model and identify which was the most appropriate one. As a result of this step, the startup owner identified the *function calling* from the OpenAI API as the feature that would better fit into the Catch Solve platform.

The second step of the technical plan was to elaborate on a TRACER BULLET [15] that explores this integration in an end-to-end example. The startup owner chose two services from the platform: (a) one that evaluated the page regarding its accessibility, (b) another that verified specific regulations from the food industry (like the fact that

allergens should always start with a capital letter). Based on that need, the TRACER BULLET implemented an agent that was connected to a Web page where a user can send prompts to interact with it. This agent used *function calling* to connect with the services. This implementation was also used to assess AH1 and AH2.

As a result of the TRACER BULLET, it was possible to confirm the feasibility of the integration of the Catch Solve platform with the Generative AI model through the *function calling*. In addition, a presentation about the implemented feature was done in a local event[2] for the software development community. During this event, not only the technical aspects but also other potential applications were discussed.

7.7.3 Hypotheses Handling

During this experience, AH1 was not accepted since, in some cases, the APIs were invoked using the wrong parameters. To overcome this, the user had to confirm the invocation of the APIs when necessary in this prototype. As a result, some additional mechanisms would need to be implemented to overcome this issue so that the feature can be incorporated into the product. For AH2, since the issues related to food industry regulations were straightforward to address, only the suggestions about the accessibility issues were evaluated. For this kind of problem, the suggestions were suitable and could be further explored with additional prompts. This hypothesis was kept open since an evaluation with other types of problems would be needed. However, this initial assessment was enough to reduce the associated uncertainty level.

Based on the implementation, the startup owner accepted hypothesis AH3, assessing that this feature is feasible by using Generative AI. However, one problem was found in this implementation: passing contextual information regarding the Web page, including its source code, in the prompts frequently surpassed the maximum number of characters. For the TRACER BULLET, a more "hard-coded" solution that could not be generalized for all cases was used to select relevant pieces of the page. Accordingly, a new hypothesis related to that was added: *AH4. It is possible to have a general solution to select relevant parts of a Web page source code to be added to the prompt.* This new hypothesis was also mapped to the business hypothesis BH5, since it would be important for the feasibility of that feature.

These initial steps reduced the general uncertainty regarding integrating Generative AI APIs with the startup software, but some hypotheses were still left open. One hypothesis was considered accepted, giving rise to a new one that captured a more specific problem. The startup owner considered it enough for its current needs since it gave him sufficient information to reason and better plan this future feature. Based on the type of API required for *function calling*, it was also possible to understand

[2] AI for Testing and Compliance, Software Developer's Thursday, https://noi.bz.it/en/events/software-developer-s-thursday/557a2b22-7b16-4d75-8270-ffcdfbf245b4.

better how to structure the software so that the impact of that future integration is reduced. Based on that, a PROTECTIVE GUIDELINE [28] was introduced to drive the system design in that direction.

7.7.4 Discussion

Based on the case study, it was possible to observe that the connection between HyMap and ArchHypo could guide the startup in identifying and handling hypotheses. In the first place, the set of hypotheses identified by HyMap captured the uncertainty that Catch Solve was experiencing from the business point of view. Further, the hypotheses that impacted the software architecture were extracted from them and managed with ArchHypo by elaborating and conducting a technical plan based on patterns suggested by the technique. The traceability among business and architectural hypotheses kept them connected, even when a new architectural hypothesis arose. Considering this evidence, combined with the previous results in adopting the techniques in isolation [24, 34, 35], it is possible to state that the proposed approach is a promising technique for software startups to manage their uncertainties.

For startups with fewer resources, it might be difficult to prioritize more technical tasks that are not converted to a new feature or to a more immediate product enhancement. The hypotheses assessment proposed by ArchHypo can support the development of a more strategic vision and the prioritization of actions. This assessment might also be useful for more mature startups with a further developed product, in which a higher number of uncertainties might arise. In this case, understanding the potential impact is important to understand where the available resources might be applied. In the end, a local adaptation of the techniques would be needed according to the startup's priorities and resources. For instance, to define the pace at which the techniques' steps should be applied would depend on the availability of resources that can be dedicated to them.

A professional with software architecture expertise is necessary for the application of the technique, since ArchHypo has a long learning curve [34] and it requires experience to choose and implement the techniques to handle the uncertainties. As future work, tool support can provide guidance and reduce the expertise needed for the use of the technique.

The presented case study had the scope of demonstrating the use of the proposed technique. However, it has some limitations. First, it inherently has a limited generalization: even though we might expect that the results would be similar in comparable contexts, we cannot guarantee that they would be valid for all startups. Second, there is a risk of confirmation bias given the involvement of the authors of this study with the techniques and the startup. To mitigate this issue, we followed the techniques as they have been previously published in peer-reviewed venues. We also described in detail, considering the limitation of space, the steps conducted in the research.

7.8 Conclusions

This chapter focused on techniques to manage uncertainty in software startups using hypotheses engineering. Uncertainty is an inherent aspect of businesses that seek innovative and disruptive solutions. Because of that, identifying and managing uncertainties is important to explore the alternatives in a more systematic way. In this direction, hypotheses engineering is a discipline that proposes the usage of hypotheses to represent this uncertainty in an explicit way. HyMap is a technique based on hypotheses engineering that uses a conceptual map to identify potential customer needs and map them to candidates of the startup software features. This map is used to elicit potential business hypotheses. ArchHypo is a technique, not exclusively focused on software startups, that uses the concepts from hypotheses engineering to manage software architecture uncertainties.

In this chapter, we proposed the combination of both techniques to provide a more complete toolset for software startups to manage their uncertainties. Based on the type of hypotheses generated by the HyMap, we proposed an approach for identifying possible impacts in the software architecture to create a bridge between them. In the case where these impacts were found, these hypotheses are assessed and managed using ArchHypo. By keeping traceability between hypotheses, it is possible for the technical side to support the business side and vice versa, building a more strategic approach to uncertainty management. The Catch Solve case study presented how this combination of both techniques would be applied in a real case. As future work, more case studies could be conducted in software startups regarding the application of both techniques combined. Based on the results, the technique can be refined and adapted to more specific contexts. Additionally, considering the strong foundation of ArchHypo in patterns, new patterns regarding the conversion of business hypotheses into architectural hypotheses can be identified and documented.

References

1. Andries, P., Debackere, K., van Looy, B.: Simultaneous experimentation as a learning strategy: business model development under uncertainty. Strateg. Entrep. J. **7**(4), 288–310 (2013). https://doi.org/10.1002/sej.1170
2. Atkinson, R.: Project management: cost, time and quality, two best guesses and a phenomenon, its time to accept other success criteria. Int. J. Project Manage. **17**(6), 337–342 (1999). https://doi.org/10.1016/S0263-7863(98)00069-6
3. Bajwa, S.S., Wang, X., Nguyen Duc, A., Abrahamsson, P.: "Failures" to be celebrated: an analysis of major pivots of software startups. Empir. Softw. Eng. **22**(5), 2373–2408 (2017). https://doi.org/10.1007/s10664-016-9458-0
4. Berg, V., Birkeland, J., Nguyen-Duc, A., Pappas, I.O., Jaccheri, L.: Software startup engineering: a systematic mapping study. J. Syst. Softw. **144**(February), 255–274 (2018). https://doi.org/10.1016/j.jss.2018.06.043
5. Bortolini, R.F., Nogueira Cortimiglia, M., Danilevicz, A.d.M.F., Ghezzi, A.: Lean Startup: a comprehensive historical review. Manage. Decis. **59**(8), 1765–1783 (2021). https://doi.org/10.1108/MD-07-2017-0663

6. Cantamessa, M., Gatteschi, V., Perboli, G., Rosano, M.: Startups' roads to failure. Sustainability **10**(7), 2346 (2018). https://doi.org/10.3390/su10072346
7. Cheng, B.H.C., Atlee, J.M., Joanne, M.: Research directions in requirements engineering. In: Proceeding FOSE '07 2007 Future of Software Engineering, pp. 285–303 (2007). https://doi.org/10.1109/FOSE.2007.17
8. Chiu, T.K.: The impact of generative AI (GenAI) on practices, policies and research direction in education: A case of chatGPT and midjourney. Interact. Learn. Environ. **32**(10), 6187–6203 (2024)
9. Choma, J., da Silva, T.S., Guerra, E.M.: Pattern Language for Lightweight and Interactive Software Analytics Process, pp. 162–189. Springer, Berlin/Heidelberg (2025). https://doi.org/10.1007/978-3-662-70810-1_4
10. Dwivedi, Y.K., Wastell, D., Laumer, S., Henriksen, H.Z., Myers, M.D., Bunker, D., Elbanna, A., Ravishankar, M.N., Srivastava, S.C.: Research on information systems failures and successes: status update and future directions. Inf. Syst. Front. **17**(1), 143–157 (2015). https://doi.org/10.1007/s10796-014-9500-y
11. Eisenhardt, K.M., Tabrizi, B.N.: Accelerating adaptive processes: product innovation in the global computer industry. Adm. Sci. Q. **40**(1), 84 (1995). https://doi.org/10.2307/2393701
12. Fabijan, A., Dmitriev, P., McFarland, C., Vermeer, L., Holmström Olsson, H., Bosch, J.: Experimentation growth: evolving trustworthy A/B testing capabilities in online software companies. J. Software: Evol. Process **30**(12), e2113 (2018). https://doi.org/10.1002/smr.2113
13. Fagerholm, F., Sanchez Guinea, A., Mäenpää, H., Münch, J.: The RIGHT model for continuous experimentation. J. Syst. Softw. **123**, 292–305 (2017). https://doi.org/10.1016/j.jss.2016.03.034
14. Frederiksen, D.L., Brem, A.: How do entrepreneurs think they create value? A scientific reflection of Eric Ries' Lean Startup approach. Int. Entrep. Manag. J. **13**(1), 169–189 (2017). https://doi.org/10.1007/s11365-016-0411-x
15. Guerra, E., Wirfs-Brock, R., Yoder, J.: Patterns for initial architectural design on agile projects. In: 4th Asian Conference on Pattern Languages of Programs (AsianPLoP) (2015)
16. Gutbrod, R., Wiele, C.: The Software Dilemma: Balancing Creativity and Control on the Path to Sustainable Software. Springer Science & Business Media, Berlin (2012)
17. Gutbrod, M., Münch, J., Tichy, M.: How do software startups approach experimentation? Empirical results from a qualitative interview study. In: Felderer, M., Méndez Fernández, D., Turhan, B., Kalinowski, M., Sarro, F., Winkler, D. (eds.) Product-Focused Software Process Improvement, pp. 297–304. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-69926-4_21
18. Herrmann, B.L., Marmer, M., Dogrultan, E., Holtschke, D.: Startup ecosystem report 2012. Telefonica Digital and Startup Genome (2012)
19. Kelly, G.: The Psychology of Personal Constructs: Volume One: Theory and Personality. Taylor & Francis, Oxfordshire (2002)
20. Lindgren, E., Münch, J.: Raising the odds of success: the current state of experimentation in product development. Inf. Softw. Technol. **77**, 80–91 (2016). https://doi.org/10.1016/j.infsof.2016.04.008
21. Melegati, J.: Towards a framework to guide the creation of development practices for software startups. In: Gregory, P., Kruchten, P. (eds.) Agile Processes in Software Engineering and Extreme Programming – Workshops, pp. 155–164. Springer International Publishing, Cham (2021)
22. Melegati, J., Wang, X., Abrahamsson, P.: Hypotheses engineering: first essential steps of experiment-driven software development. In: IEEE/ACM Joint 4th International Workshop on Rapid Continuous Software Engineering and 1st International Workshop on Data-Driven Decisions, Experimentation and Evolution (RCoSE/DDrEE), pp. 16–19 (2019). https://doi.org/10.1109/RCoSE/DDrEE.2019.00011
23. Melegati, J., Edison, H., Wang, X.: XPro: A model to explain the limited adoption and implementation of experimentation in software startups. IEEE Trans. Softw. Eng. **48**(6), 1929–1946 (2022). https://doi.org/10.1109/TSE.2020.3042610

24. Melegati, J., Guerra, E., Wang, X.: Hymap: eliciting hypotheses in early-stage software startups using cognitive mapping. Inf. Softw. Technol. **144**, 106807 (2022)
25. Mohanani, R., Ram, P., Lasisi, A., Ralph, P., Turhan, B.: Perceptions of creativity in software engineering research and practice. In: 2017 43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 210–217 (2017). https://doi.org/10.1109/SEAA.2017.21
26. Nguyen-Duc, A., Cabrero-Daniel, B., Przybylek, A., Arora, C., Khanna, D., Herda, T., Rafiq, U., Melegati, J., Guerra, E., Kemell, K.K., et al.: Generative artificial intelligence for software engineering–a research agenda. Preprint. arXiv:2310.18648 (2023)
27. Olsson, H.H., Bosch, J.: From opinions to data-driven software R&D: a multi-case study on how to close the 'open loop' problem. In: 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications, pp. 9–16. IEEE, Piscataway (2014)
28. Paris, A., Guerra, E., Silveira, F., Silva, K.: Patterns for small adjustments in the development process to deal with architectural uncertainties. In: Proceedings of the 29th European Conference on Pattern Languages of Programs, People, and Practices, pp. 1–8 (2024)
29. Paternoster, N., Giardino, C., Unterkalmsteiner, M., Gorschek, T., Abrahamsson, P.: Software development in startup companies: a systematic mapping study. Inf. Softw. Technol. **56**(10), 1200–1218 (2014). https://doi.org/10.1016/j.infsof.2014.04.014
30. Ralph, P., Kelly, P.: The dimensions of software engineering success. In: Proceedings of the 36th International Conference on Software Engineering, pp. 24–35. ACM, New York (2014). https://doi.org/10.1145/2568225.2568261
31. Semujanga, B., Mikalef, P.: Exploring the productivity impacts of generative AI in organizations. In: Conference on e-Business, e-Services and e-Society, pp. 103–111. Springer, Berlin (2024)
32. Shenhar, A.J., Dvir, D., Levy, O., Maltz, A.C.: Project success: a multidimensional strategic concept. Long Range Plan. **34**(6), 699–725 (2001). https://doi.org/10.1016/S0024-6301(01)00097-8
33. Silva, K., Adolfo, L.B., Coppe, A., Silveira, F.F., Ferreira, M., Guerra, E.: Patterns for using hypothesis engineering to manage architectural uncertainties. In: Proceedings of the 29th European Conference on Pattern Languages of Programs, People, and Practices, pp. 1–8 (2024)
34. Silva, K., Melegati, J., Silveira, F., Wang, X., Ferreira, M., Guerra, E.: Archhypo: managing software architecture uncertainty using hypotheses engineering. IEEE Trans. Softw. Eng. **51**(2), 430–448 (2024)
35. Silva, K., Melegati, J., Wang, X., Ferreira, M., Guerra, E.: Using hypotheses to manage technical uncertainty and architecture evolution in a software start-up. IEEE Softw. **41**(4), 7–13 (2024)
36. Unterkalmsteiner, M., Abrahamsson, P., Nguyen-duc, A., Baltes, G.H., Conboy, K., Dennehy, D., Sweetman, R., Edison, H., Shahid, S., Wang, X., Garbajosa, J., Gorschek, T., Hokkanen, L., Lunesu, I., Marchesi, M., Morgan, L., Selig, C., Oivo, M., Shah, S., Kon, F.: Software startups - a research agenda. e-Informatica Softw. Eng. J. **10**(1), 1–28 (2016). https://doi.org/10.5277/e-Inf160105
37. Wirfs-Brock, R., Yoder, J., Guerra, E.: Patterns to develop and evolve architecture during an agile software project. In: Proceedings of the 22nd Conference on Pattern Languages of Programs, pp. 1–18 (2015)
38. Woods, E., Erder, M., Pureur, P.: Continuous Architecture in Practice: Software Architecture in the Age of Agility and DevOps. Addison-Wesley Professional, Boston (2021)